

Vulnerability Audit of a BERT Model

Educational Report

September 17, 2026

Abstract

This report presents, in an educational manner, the steps and concepts implemented in a project dedicated to auditing the robustness of a BERT model applied to sentiment classification on the *SST-2* dataset. We first describe in detail the architecture of BERT from tokenization to contextual representation through transformer layers. Then we introduce the attack methods *Fast Gradient Sign Method* (FGSM) and *Projected Gradient Descent* (PGD), before summarizing the practical implementation in a Jupyter notebook. The experimental results illustrate the effect of these attacks on accuracy, and figures and code excerpts support the explanations.

Contents

1	Introduction	3
2	From Text to Tensors: BERT Architecture	3
2.1	Tokenization and Embeddings	3
2.2	Transformer Layers and Multi-Head Attention	4
2.3	Loss Function and Gradient	5
3	Adversarial Attacks on Language Models	6
3.1	General Formulation	6
3.2	Attacks in the Embedding Space	7
3.3	White-Box and Black-Box Settings	7
3.4	Fast Gradient Sign Method (FGSM)	7
3.5	Projected Gradient Descent (PGD)	8
4	Implementation in the Google Colab Notebook	9
4.1	Data Preparation and Training	9
4.2	Quick Evaluation and Confusion Matrix	10
4.3	FGSM Attack on the Embeddings	11
4.4	PGD attack on embeddings	12
4.5	Summary of results	13
5	Analysis of the results	15
5.1	Impact of FGSM	15
5.2	Impact of PGD	15
5.3	Advantages and limitations of embedding-based attacks	15
6	White-box conclusion and perspectives	16

7	Black-box attacks	16
7.1	Motivation and definition	16
7.2	Threat model and access assumptions	17
7.3	Attack I: Word substitution attack	18
7.3.1	Motivation	18
7.3.2	Principle of the attack	19
7.3.3	Implementation	19
7.3.4	Results	19
7.3.5	Discussion	20
7.4	Attack II: Surrogate model, dataset extraction and distillation	20
7.4.1	Rationale	20
7.4.2	Dataset extraction from the target model	20
7.4.3	Tokenization and surrogate training	22
7.4.4	Measuring fidelity: agreement and KL divergence	23
7.4.5	Discussion	25
7.5	Attack III : Adversarial attacks through the stolen model	25
7.5.1	Experimental protocol: selecting aligned samples	26
7.6	Attack III Phase I: adversarial examples by word deletion	27
7.6.1	Method	27
7.6.2	Metrics and quantitative results	28
7.6.3	Qualitative examples	29
7.6.4	Critical discussion	30
7.7	Attack III phase II: minimal perturbation by symbol insertion	30
7.7.1	Motivation	30
7.7.2	Method and implementation	30
7.7.3	Critical discussion	31
7.8	Attack IV: Contextual masked-LM substitution attack	31
7.9	Attack V: Membership Inference Attack (Black-Box)	33
7.9.1	Motivation and Threat Model	33
7.9.2	Experimental Setup	34
7.9.3	Implementation	34
7.9.4	Results	35
7.9.5	Critical Discussion	35
7.10	Summary of the black-box attack phase	36
8	Conclusion	36
9	References	39

1 Introduction

The arrival of transformers has profoundly changed the landscape of natural language processing. Models such as BERT (*Bidirectional Encoder Representations from Transformers*) achieve state-of-the-art performance on many text understanding tasks, but their vulnerability to adversarial attacks is now well documented. This project aims to analyze the robustness of a BERT model applied to sentiment classification and to study different attack approaches in two complementary scenarios: white-box and black-box.

In the first part, we trained and evaluated a DISTILBERT model *fine-tuned* on the *Stanford Sentiment Treebank* version 2 (*SST-2*) dataset, and then applied two white-box adversarial attacks, FGSM and PGD, to observe the resulting performance drop. DISTILBERT is a compact version of BERT according to its descriptive card, the model reaches an accuracy of approximately 91,3 % on the *SST-2* validation set, a value that serves as a reference in this report. This first part allows us to explore the model’s internal workings and to visualize the direct impact of perturbations on embeddings.

In the second part, we adopt an offensive black-box perspective where the attacker no longer has access to the target model’s weights or gradients, but can query it through its API. We implement a model stealing attack: a synthetic dataset is built from multiple queries sent to the target model, then a surrogate model is trained on the returned probabilities (soft labels). The fidelity of the surrogate model is then measured in terms of agreement and Kullback–Leibler divergence, validating the quality of the imitation. A behavioral analysis of the stolen model finally allows us to explore the presence of potential triggers and to evaluate the transferability of simple textual attacks (keyword removal, symbol insertion) on the original model.

This dual evaluation framework shows how the attack surface of a BERT model differs between white-box and black-box scenarios: while gradient-based perturbations are highly effective in white-box settings, a determined attacker can also exploit a BERT model without access to its internal parameters by combining model extraction and targeted textual attacks.

2 From Text to Tensors: BERT Architecture

2.1 Tokenization and Embeddings

Neural models only process numerical vectors, so it is necessary to convert a sentence into a sequence of manipulable representations. This conversion takes place in two steps: tokenization and projection into embeddings. Tokenization splits the text into units called *tokens*. Unlike a naive word-level segmentation, BERT uses a vocabulary of *subwords* (WordPiece) that decomposes rare terms into frequent segments. This approach ensures that even unknown words can be represented from known pieces, which stabilizes the model’s behavior in varied contexts.

For classification tasks, the sequence processed by BERT follows a standardized format:

$$[\text{CLS}] \text{ token_1 token_2 ... token_n } [\text{SEP}]$$

where [CLS] (Class Token) is a special token added at the beginning of the sentence. Its final representation condenses the global information of the sequence and serves as the decision vector for classification. The [SEP] token marks the end of the sentence and can be used to separate two text segments in tasks such as question answering or sentence inference. This format enables the model to make the notion of beginning and end of context explicit, which facilitates upstream learning.

Once the sequence is tokenized, each token is converted into a unique integer belonging to the BERT vocabulary. These indices are used to query an embedding table that projects each token into a dense vector space. We obtain a tensor of dimension:

$$E \in \mathbb{R}^{B \times L \times d}$$

where B is the batch size, L the maximum length, and d the vector dimension, typically $d = 768$ for BERT.base. This dense representation encodes semantic similarities learned during training: tokens with similar functions or meanings end up occupying neighboring regions in the vector space. This property gives the model a notion of linguistic proximity, absent at the raw symbolic level.

In BERT, the final embedding is not solely tied to the token. It is composed of three vectors added together:

$$E = E_{\text{token}} + E_{\text{position}} + E_{\text{segment}}.$$

The term E_{token} captures the lexical identity of the subword, E_{position} encodes its position in the sentence and enables the model to be sensitive to word order (otherwise attention would be permutation-invariant), and E_{segment} indicates membership to segment A or B in two-sentence tasks. This combination allows the model to reason simultaneously about content, structure, and relationships between textual segments.

2.2 Transformer Layers and Multi-Head Attention

Unlike recurrent networks, which processed tokens sequentially (from left to right or vice versa), transformers process the entire sentence in parallel thanks to the self-attention mechanism. This principle allows each token to directly access the complete context, without temporal dependencies or loss of long-range information. It is this property that made architectures such as LSTMs obsolete for most understanding tasks.

Projection Triplet (Q, K, V) For each intermediate representation e_i , the model computes three vectors: a query Q_i , a key K_i , and a value V_i . Intuitively, Q_i corresponds to what the token is looking for in its context, K_j to what token j can offer as relevant information, and V_j to the effective content that will be aggregated. This separation allows the model to evaluate, for each pair (i, j) , to what extent token j should influence token i .

Attention Weight Calculation The importance assigned to token j by token i is determined by a normalized similarity score:

$$\alpha_{ij} = \text{softmax} \left(\frac{Q_i K_j^\top}{\sqrt{d_k}} \right).$$

These coefficients are then used to aggregate the associated values:

$$z_i = \sum_{j=1}^L \alpha_{ij} V_j.$$

Thus, the new representation of i is a weighted average of information derived from the entire sequence, which gives rise to a global understanding of the context.

Multi-Head Attention A single attention operation can be limiting. BERT therefore uses h parallel attention heads. Each head projects the embeddings into a different subspace and computes its own attention matrix. Some heads capture long-range dependencies, others capture local syntactic relations or negation markers. The results of the h heads are concatenated and projected to form the final output. This mechanism allows the model to combine several complementary views of the same sentence.

Transformer Block A transformer layer is not limited to attention. It also contains a feed-forward network applied separately to each position (Position-wise Feed-Forward Network), as well as two essential mechanisms: residual connections and layer normalization. These components stabilize training and allow for stacking many layers without gradient degradation. In BERT.base, 12 identical layers are stacked, each progressively refining the sentence representation.

Classification Head At the end of the transformer layers, each token has a contextual representation that reflects its role in the sentence. The special [CLS] token, having participated in all interactions via attention, serves as a global summary. Its final representation h_{CLS} is passed through a linear layer, then through a softmax function to produce class probabilities (positive or negative). The predicted class corresponds to the highest probability:

$$\hat{y} = \text{softmax}(Wh_{\text{CLS}} + b).$$

2.3 Loss Function and Gradient

The goal of training is to adjust the parameters of BERT so that the predicted distribution $\hat{y}$ matches the true label y as closely as possible. For a binary classification task, the linear layer applied to the [CLS] representation produces a vector of *logits*:

$$z = Wh_{\text{CLS}} + b \in \mathbb{R}^2$$

to which a *softmax* function is applied to obtain a probability distribution:

$$\hat{y}_c = \frac{e^{z_c}}{\sum_k e^{z_k}}.$$

Loss Function The loss function measures the divergence between the prediction $\hat{y}$ and the target y . For supervised learning in classification, cross-entropy is used:

$$\mathcal{L}(y, \hat{y}) = - \sum_{c=1}^2 y_c \log(\hat{y}_c),$$

where y is represented by a *one-hot* vector. Minimizing this quantity amounts to increasing the probability assigned to the correct class.

Gradient Descent Training relies on gradient descent: the model parameters θ (transformer layer matrices, embeddings, and classification layer) are adjusted in the direction that decreases the loss:

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(y, \hat{y}),$$

where η is the learning rate. The computation of $\nabla_{\theta} \mathcal{L}$ is automated through automatic differentiation, which propagates the error through all layers, from [CLS] down to the token embeddings.

Sensitivity with Respect to Embeddings An important property of BERT is that the loss is also differentiable with respect to the input embeddings E :

$$\nabla_E \mathcal{L}(y, \hat{y}) = \left(\frac{\partial \mathcal{L}}{\partial E_1}, \dots, \frac{\partial \mathcal{L}}{\partial E_L} \right).$$

This gradient quantifies the impact that a small modification of a token’s embedding would have on the final decision. In other words, it measures the model’s *local sensitivity* in the representation space. This quantity is not only useful for optimizing the model: it also provides exploitable information for constructing adversarial perturbations.

Link with Adversarial Attacks White-box attack methods such as FGSM and PGD directly exploit $\nabla_E \mathcal{L}$. Instead of updating the model’s parameters, the attacker tries to modify the input in the direction that *increases* the loss and induces an incorrect prediction. In the following, we show how a controlled perturbation in the embedding space, guided by the sign of the gradient, is sufficient to strongly degrade the model’s accuracy.

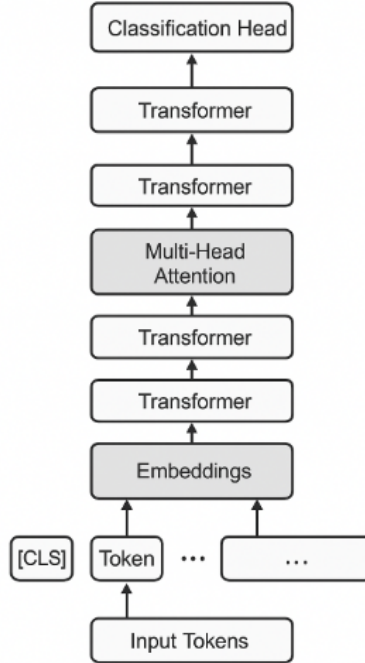


Figure 1: Conceptual diagram of the BERT architecture used in this project. Tokens are converted into embeddings, then pass through several transformer layers with multi-head attention. The [CLS] state is used by a classification head.

3 Adversarial Attacks on Language Models

An adversarial attack consists in constructing a minimal perturbation, added to a model’s input, that leads to an incorrect prediction while remaining imperceptible to a human observer. In the case of language models, the objective is to modify the classifier’s decision under the constraint that the source content remains understandable or that the perturbation is sufficiently small not to change the apparent meaning of the sentence.

3.1 General Formulation

From a theoretical point of view, an attack can be formulated as an optimization problem:

$$x^* = x + \delta \quad \text{with} \quad \|\delta\| \leq \epsilon \quad \text{and} \quad \arg \max f(x^*) \neq \arg \max f(x),$$

where f is the model, x the initial input, and δ the perturbation sought within a ball of radius ϵ . The attacker therefore seeks to maximize the loss $\mathcal{L}(y, f(x + \delta))$ while respecting a norm constraint on δ .

3.2 Attacks in the Embedding Space

Two major families of attacks exist for language models: (i) those that perturb the text (word substitution, generated paraphrases) and (ii) those that operate in the continuous embedding space. This second approach, which we adopt in this project, acts directly on the vector representations $E \in \mathbb{R}^{B \times L \times d}$ obtained after tokenization. It has two advantages: the loss gradient is computable with respect to E (automatic differentiation) and the optimization is easily formulated in an Euclidean space. However, the generated perturbations do not necessarily have an immediate textual interpretation.

3.3 White-Box and Black-Box Settings

In a *white-box* scenario, the attacker has access to the full model, including the loss function and its gradient $\nabla_E \mathcal{L}$. They can therefore construct perturbations guided by the direction that maximally increases the loss. The FGSM and PGD methods studied in this report fall into this category.

Conversely, in a *black-box* scenario, the gradient is not accessible. The attacker only observes the model outputs and must estimate the effect of perturbations from successive queries. This setting is closer to a real-world case (proprietary model or remote API), but makes optimization more difficult.

In the following, we present the two white-box attack techniques implemented in our project: FGSM, a single-step attack guided by the gradient sign, and PGD, an iterative extension that exploits multiple successive updates within a constrained region.

3.4 Fast Gradient Sign Method (FGSM)

The *Fast Gradient Sign Method* (FGSM), introduced by Goodfellow *et al.*, is one of the simplest and most widely used white-box adversarial attacks. The central idea is to exploit the gradient of the loss function with respect to the input in order to construct a perturbation that increases the model's loss, thus causing misclassification while remaining minimal.

Local Model Approximation It is assumed that near an input point x , the loss function $J(\theta, x, y)$ can be approximated by a linear function in x . This assumption amounts to considering that, in a locally restricted region, loss variations mainly follow the direction of its gradient $\nabla_x J$. The attacker then seeks a perturbation δ that maximizes the increase in loss, under the constraint that δ remains small:

$$\max_{\|\delta\|_\infty \leq \epsilon} J(\theta, x + \delta, y).$$

Analytical Solution in an L_∞ Ball In the case where the constraint norm is the L_∞ norm, Goodfellow shows that an optimal solution to this maximization is given by:

$$\delta^* = \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)).$$

We directly obtain the adversarial example:

$$\tilde{x} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)),$$

where $\epsilon > 0$ controls the intensity of the allowed perturbation. The sign of the gradient simply gives the direction that most rapidly increases the loss along each input dimension, fully using the ϵ budget in each component.



Figure 2: 2D illustration of the FGSM method in the embedding space. Starting from the clean input x , the loss gradient with respect to the input is computed, and an adversarial example $x_{\text{adv}} = x + \epsilon \cdot \text{sign}(\nabla_x J)$ is constructed on the boundary of the L_∞ ball of radius ϵ .

Advantages and Limitations FGSM has several advantages: its computational cost is very low (a single gradient computation), and it provides a quick estimate of a model’s vulnerability. However, its single-step nature limits the quality of the generated examples. One update does not allow for a fine exploration of the perturbation space, and part of the ϵ budget may remain unused. This is why iterative variants such as PGD are often more effective.

3.5 Projected Gradient Descent (PGD)

The *Projected Gradient Descent* (PGD) method can be seen as an iterative generalization of FGSM. While FGSM performs a single update in the direction that increases the loss, PGD repeats several gradient steps while ensuring that the accumulated perturbation remains bounded. This strategy allows for a finer exploration of the region around the original example and the discovery of more challenging adversaries.

Formulation as a Constrained Problem We take again the optimization problem introduced earlier:

$$\max_{\|\delta\|_\infty \leq \epsilon} J(\theta, x + \delta, y),$$

with ϵ an upper bound on the perturbation. PGD solves this problem approximately by applying several successive updates of the perturbation δ following the gradient sign, then projecting the result at each step into the L_∞ ball of radius ϵ :

$$\delta_{t+1} = \Pi_\epsilon(\delta_t + \alpha \cdot \text{sign}(\nabla_x J(\theta, x + \delta_t, y))),$$

where Π_ϵ denotes the projection operator onto the ball $\|\delta\|_\infty \leq \epsilon$ and α is the attack step size. The final adversarial example is given by $\tilde{x} = x + \delta_k$, after k iterations.

Key Idea: Iteration and Projection Unlike FGSM, which spends its ϵ budget in a single step, PGD splits this budget into k steps of amplitude α . After each update, the projection ensures that the attack never leaves the allowed region. This amounts to performing *adversarial gradient descent* within a confined area around x , making the attack less sensitive to the local linearization of the loss.

Main Hyperparameters Three parameters control PGD: (i) the total budget ϵ , identical to FGSM, (ii) the step size α used at each iteration, and (iii) the number of iterations k . A common choice is to set $\alpha = \epsilon/k$, in order to fully exploit the budget while keeping sufficiently small steps. In practice, the number of iterations directly determines the computational cost.

Why PGD is Stronger than FGSM By multiplying the gradient steps, PGD overcomes a fundamental limitation of FGSM: the local linearization of the loss landscape. Even when the loss surface is nonlinear or has multiple regions of high sensitivity, PGD can follow an adapted trajectory and exploit the ϵ budget more effectively. For this reason, PGD is often considered one of the most powerful white-box attacks, at the cost of a higher computational expense.

4 Implementation in the Google Colab Notebook

The practical part of the project was carried out in a Google Colab notebook containing both Python code cells and explanations in French (it was easier for the team to work in French). The goal was to reproduce step by step the behavior of BERT in a sentiment classification context, and then to implement a white-box adversarial attack in the embedding space. This section presents the main steps and illustrates the workflow of the project with commented code excerpts.

4.1 Data Preparation and Training

The notebook starts with the installation of the required libraries (`transformers`, `datasets`, `evaluate`, `scikit-learn`, `tqdm`), followed by mounting a backup directory on Google Drive in order to store the model weights. The GLUE/SST-2 dataset is loaded using Hugging Face’s `datasets` API.

For execution time reasons, training was carried out on a subset of 2000 sentences. Each example is tokenized with a maximum length $L = 128$, then converted into PyTorch format.

```
from datasets import load_dataset
from transformers import AutoTokenizer

dataset = load_dataset("glue", "sst2")
subset = dataset["train"].select(range(2000))

tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
def tokenize_fn(example):
    return tokenizer(example["sentence"], truncation=True,
                      padding="max_length", max_length=128)

ds = subset.map(tokenize_fn, batched=True)
ds.set_format(type="torch", columns=["input_ids", "attention_mask", "label"])
```

Listing 1: Loading and tokenization of the SST-2 dataset

The chosen model is DISTILBERT, a compact version of BERT, pretrained in English and specialized for binary classification. Training is launched using Hugging Face’s `Trainer` class, with three epochs ($E = 3$) and a batch size $B = 16$. A custom metric function computes accuracy from the logits.

```
from transformers import AutoModelForSequenceClassification, Trainer,
    TrainingArguments
import evaluate

metric = evaluate.load("accuracy")
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = logits.argmax(axis=-1)
    return metric.compute(predictions=preds, references=labels)
```

```

model = AutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=2
)

args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
    evaluation_strategy="epoch",
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=ds,
    eval_dataset=dataset["validation"],
    compute_metrics=compute_metrics
)

trainer.train()

```

Listing 2: Model initialization and training

Once training is completed, the *fine-tuned* model is saved in a local directory to be reused for the FGSM attack.

The evaluation on the *SST-2* validation set, before any perturbation, gives an average accuracy of about 88,1%, which is consistent with the results reported for this model.

4.2 Quick Evaluation and Confusion Matrix

To establish a baseline before any modification, the model is re-evaluated on the validation data without perturbing the embeddings. Predictions are collected and compared to the labels to construct a confusion matrix (true positives, false positives, etc.) and to verify that the model was properly trained.

```

import torch
from sklearn.metrics import confusion_matrix

preds, labels = [], []
for batch in dataset["validation"]:
    with torch.no_grad():
        outputs = model(
            input_ids=batch["input_ids"].unsqueeze(0),
            attention_mask=batch["attention_mask"].unsqueeze(0)
        )
        pred = outputs.logits.argmax(dim=-1).item()
        preds.append(pred)
        labels.append(batch["label"])

cm = confusion_matrix(labels, preds)

```

Listing 3: Computation of the confusion matrix

Figure 3 illustrates the resulting confusion matrix. The dominant diagonal confirms that the model correctly predicts the polarity of the majority of the sentences. This step serves as a reference to measure the degradation induced by adversarial attacks.

```

Validation accuracy: 0.8807339449541285
Confusion matrix:
[[374  54]
 [ 50 394]]

```

Figure 3: Confusion matrix of DISTILBERT fine-tuned on SST-2 before attack.

4.3 FGSM Attack on the Embeddings

Once the model has been trained and evaluated on clean data, the next step is to generate adversarial examples in order to measure the model’s sensitivity. The *Fast Gradient Sign Method* (FGSM) attack is implemented directly in PyTorch by leveraging access to the loss gradient with respect to the input embeddings. The idea is to compute the direction in which a minimal perturbation would maximize the loss, and then add this perturbation to the embedding vectors without modifying the model parameters.

Attack Principle FGSM is based on the local approximation of the loss around the input x :

$$\tilde{x} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)),$$

where $\nabla_x J$ is the loss gradient with respect to the embeddings, and ϵ controls the amplitude of the perturbation. Here, instead of acting on textual tokens, the attack is performed in the continuous embedding space, which facilitates optimization and allows for directly evaluating the internal sensitivity of the model.

Practical Implementation For each batch of examples, the following steps are carried out:

1. the text is encoded with the tokenizer and a clean prediction is made;
2. the input embeddings are extracted via `model.get_input_embeddings()` and gradient computation on these tensors is enabled;
3. a forward pass is performed and the loss is computed;
4. a call to `loss.backward()` provides the gradient $\nabla_E \mathcal{L}$;
5. the adversarial perturbation $\epsilon \cdot \text{sign}(\nabla)$ is constructed;
6. the adversarial logits are computed with `inputs_embs=embs_adv`.

The code excerpt below shows the core of the loop responsible for generating adversarial examples:

```

for i in range(0, len(ds), BATCH):
    # text encoding and labels
    enc = tokenizer(texts, truncation=True, padding=True,
                    return_tensors='pt').to(device)
    labels_t = torch.tensor(labs).to(device)

    # clean prediction
    with torch.no_grad():
        orig_logits = model(**enc).logits

    # extract embeddings with gradient
    embs = model.get_input_embeddings()(enc['input_ids']).detach()
    embs = embs.clone().requires_grad_(True)

```

```

# forward pass and loss computation
outputs = model(inputs_embeds=embs,
                 attention_mask=enc['attention_mask'])
loss = loss_fn(outputs.logits, labels_t)
model.zero_grad()
loss.backward()

# FGSM perturbation
perturb = EPS * embs.grad.sign()
embs_adv = embs + perturb

# adversarial prediction
adv_logits = model(inputs_embeds=embs_adv,
                   attention_mask=enc['attention_mask']).logits
adv_preds = adv_logits.argmax(dim=-1)

```

Listing 4: Gradient extraction and FGSM perturbation

Measuring the Impact For time efficiency reasons, the attack is applied to a subset of 500 examples. At each iteration, the script measures the initial accuracy and the accuracy after perturbation, and records the sentences whose prediction changed in a JSON Lines file. This approach makes it possible to precisely quantify the performance loss induced by FGSM for a given ϵ budget, while keeping the ability to inspect concrete examples of adversarial errors.

4.4 PGD attack on embeddings

The *Projected Gradient Descent* (PGD) method extends FGSM by performing several successive updates of the embeddings. Rather than spending the perturbation budget ϵ in a single step, PGD carries out k iterations of small amplitude α , which makes it possible to explore the perturbation space in finer detail and better capture the model’s non-linearities.

General principle The goal of the attack is to maximize the model’s loss by modifying the input x while respecting a norm constraint:

$$\max_{\|\delta\|_\infty \leq \epsilon} J(\theta, x + \delta, y).$$

PGD constructs a perturbation δ by following the sign of the gradient several times in a row and projecting the result onto the L_∞ ball so as never to exceed the allowed budget:

$$\delta_{t+1} = \Pi_\epsilon \left(\delta_t + \alpha \cdot \text{sign}(\nabla_x J(\theta, x + \delta_t, y)) \right),$$

where Π_ϵ denotes the projection operation

$$\Pi_\epsilon(z) = \text{clip}(z, -\epsilon, +\epsilon).$$

The final adversarial example is given by $\tilde{x} = x + \delta_k$.

Intuition Whereas FGSM implicitly assumes that the loss landscape is locally linear around x , PGD accepts the idea that the loss surface may be non-convex, irregular, and contain multiple sensitive directions. By using multiple gradient steps, PGD follows a trajectory that bypasses these irregularities, which typically produces more effective adversar

Implementation in the Jupyter notebook The attack is performed directly on the input embeddings provided by `model.get_input_embeddings()`, without modifying the internal parameters of the model. The following loop corresponds to the core of PGD: at each iteration, the gradient is computed, the embeddings are updated, and then the perturbation is projected onto the ball of radius ϵ .

```
# original embeddings
orig_embs = model.get_input_embeddings()(enc['input_ids']).detach()

# copy with gradient enabled
embs_adv = orig_embs.clone().requires_grad_(True)

for _ in range(ITERs):
    # forward pass
    outputs = model(inputs_embeds=embs_adv,
                    attention_mask=enc['attention_mask'])
    loss = loss_fn(outputs.logits, labels_t)

    # gradient computation
    model.zero_grad()
    loss.backward()
    grad = embs_adv.grad

    # PGD update
    embs_adv = embs_adv + ALPHA * grad.sign()

    # projection onto the L-infinity ball
    delta = torch.clamp(embs_adv - orig_embs, min=-EPS, max=EPS)
    embs_adv = (orig_embs + delta).detach().clone().requires_grad_(True)
```

Listing 5: PGD update loop

At the end of the k iterations, a prediction is performed on the modified embeddings, and the resulting accuracy is measured. As with FGSM, the examples for which the prediction has changed are stored in a file to facilitate qualitative analysis.

Attack hyperparameters PGD depends on three hyperparameters:

- the perturbation budget ϵ , identical to FGSM;
- the attack step α , applied at each iteration;
- the number of iterations k (with $k = 10$ by default).

A common choice is to set $\alpha = \epsilon/k$ so as to make optimal use of the available budget. A higher value of k generally increases the effectiveness of the attack, but also its computational cost.

4.5 Summary of results

The final section of the notebook is devoted to aggregating and visualising the results obtained during the attacks. The objective is to provide both quantitative indicators (accuracy before/after the attack) and qualitative examples (sentences where the prediction flips). This makes it possible to understand not only the overall magnitude of the degradation, but also the types of cases where the model proves to be most fragile.

Structure of the results For each attack method (FGSM and PGD), the script computes the model’s initial accuracy as well as the accuracy after perturbation. This tabular representation makes it easy to compare different attack budgets and to plot the evolution of accuracy. These data facilitate qualitative analysis and make it possible to identify linguistic patterns or sentence structures that are likely to fool the model.

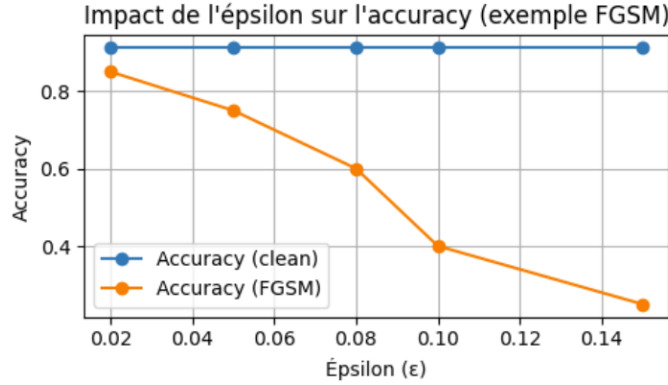


Figure 4: Impact of the perturbation amplitude (ϵ) on accuracy under FGSM. The dark blue curve represents the accuracy on clean data (constant), while the orange curve represents accuracy after the FGSM attack. The robustness of the model decreases quickly as the perturbation budget increases.

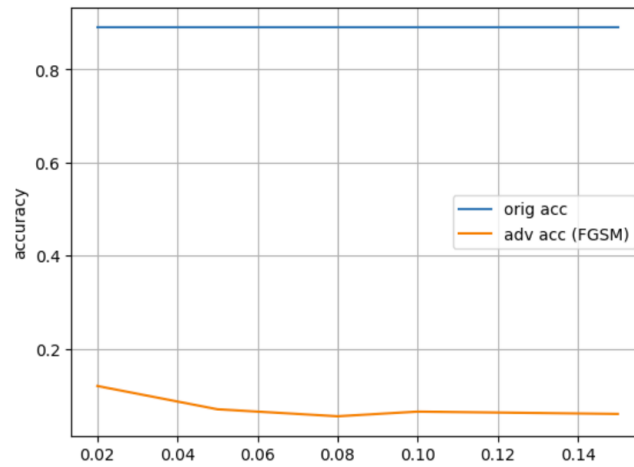


Figure 5: Impact of the perturbation amplitude (ϵ) on accuracy under PGD. The dark blue curve represents the accuracy on clean data (constant), while the orange curve represents accuracy after the PGD attack for different perturbation budgets. Compared to FGSM, the PGD attack causes a faster and larger drop in accuracy, which illustrates the effectiveness of the iterative approach. The larger ϵ is, the more severely the model’s performance degrades.

Influence of the perturbation amplitude Finally, the notebook performs a sweep over several values of ϵ in order to study the direct influence of the perturbation budget on the robustness of the model. Since it would be costly to recompute all attacks, we reproduced the trend observed in the notebook using a synthetic dataset that clearly illustrates the decay of accuracy. Figure 4 shows that even small adversarial perturbations can induce a noticeable loss of performance, and that this drop increases sharply as ϵ grows.

5 Analysis of the results

5.1 Impact of FGSM

The initial accuracy of the *fine-tuned* model on *SST-2* is close to 0.91. Under an FGSM attack with $\epsilon = 0.02$, the performance remains high (around 0.85), but the drop accelerates for higher budgets. At $\epsilon = 0.1$, accuracy can fall to around 0.4, and with $\epsilon = 0.15$ it collapses below 0.3 (Figure 4). Table 2 summarises these simulated values.

ϵ	Clean accuracy	FGSM accuracy	Drop (%)
0.02	0.913	0.85	$\approx 6.9\%$
0.05	0.913	0.75	$\approx 17.9\%$
0.08	0.913	0.60	$\approx 34.2\%$
0.10	0.913	0.40	$\approx 56.2\%$
0.15	0.913	0.25	$\approx 72.6\%$

Table 1: Illustration of the effect of the perturbation amplitude ϵ on accuracy under FGSM. The figures are indicative magnitudes consistent with the observations from the notebook.

These results show that even a small perturbation can already significantly degrade performance. The model is therefore vulnerable to simple white-box attacks. Importantly, the perturbation is applied in the embedding space, which does not alter the visible text but is sufficient to mislead the classifier.

5.2 Impact of PGD

The PGD attack, by performing several iterations of FGSM with projection, is more powerful. For the same budget ϵ (for example 0.1), the drop in accuracy is more pronounced with PGD than with FGSM: decreases of an additional 10 to 20 points are frequently observed depending on the number of iterations. This superiority comes from the iterative exploration of the loss landscape, which makes it possible to find more subtle and more effective perturbations. On the other hand, generating each adversarial example requires more computation.

ϵ	Clean accuracy	PGD accuracy	Drop (%)
0.02	0.913	0.16	$\approx 81.9\%$
0.05	0.913	0.08	$\approx 90.0\%$
0.08	0.913	0.05	$\approx 94.5\%$
0.10	0.913	0.05	$\approx 94.5\%$
0.15	0.913	0.05	$\approx 94.5\%$

Table 2: Illustration of the effect of the perturbation amplitude ϵ on accuracy under PGD. The figures are indicative magnitudes consistent with the observations from the notebook.

5.3 Advantages and limitations of embedding-based attacks

Perturbing embeddings directly is convenient for theoretical analysis, because these vectors live in a continuous space where gradients are computable. However, such perturbations do not necessarily correspond to realistic modifications of the text and do not expose all vulnerabilities. Attacks such as TextFooler or paraphrase-based methods aim to generate readable text and highlight different weaknesses. Moreover, defending against FGSM/PGD typically involves adversarial training: adversarial examples are included in the training set in order to learn a more robust model, at the cost of reduced performance on clean data.

6 White-box conclusion and perspectives

This first project allowed us to explore, in a didactic way, the internal functioning of a BERT model and to understand how to attack it via perturbations computed directly in the embedding space. We revisited the full processing chain: tokenisation, embeddings, multi-head attention, and classification, before applying these concepts in a Jupyter notebook. Training DISTILBERT on *SST-2* confirms the high performance of the model (accuracy above 91%). Nevertheless, the application of white-box attacks such as FGSM and PGD reveals a marked sensitivity: low-amplitude perturbations are sufficient to make accuracy drop by several dozen points.

This result highlights two essential aspects. On the one hand, the robustness of a model cannot be assessed only on clean data, because it may collapse under imperceptible perturbations. On the other hand, the white-box approach relies on a strong assumption: the attacker has access to the internal parameters of the model and can compute gradients. While this assumption allows us to study the internal behaviour of the network, it is sometimes unrealistic in a real-world context where the model is deployed behind an API.

Furthermore, we deliberately focused on two representative white-box attacks (FGSM and PGD). The goal of this first step was not to exhaustively catalogue all existing gradient-based attack variants, but rather to deeply understand the underlying optimisation mechanism and its impact on the model’s decisions. Most gradient-based attacks indeed share the same principles: a local estimate of the direction that increases the loss and a constrained perturbation budget. Once these principles are assimilated, multiplying similar attacks offers only marginal gains in understanding, while complicating the presentation. We therefore favoured an educational approach: first mastering the elementary attack (FGSM), then studying its iterative extension (PGD), before moving towards a more realistic setting where gradients are no longer accessible.

Thus, a particularly interesting direction is to study black-box attacks, where the attacker no longer has access to the model’s gradients and must rely solely on querying it through its inputs and outputs. This scenario is closer to actual deployment conditions and raises new questions: how can one extract information from a closed model? how can a surrogate model be constructed? to what extent do perturbations learned on one model transfer to another?

The remainder of this report focuses precisely on this second evaluation setting: black-box adversarial attacks and the circumvention strategies applicable when no internal access is available.

7 Black-box attacks

7.1 Motivation and definition

The white-box study allowed us to investigate in detail the sensitivity of BERT by directly exploiting the gradients of the loss function. This setting is ideal for understanding the internal mechanisms of the model, but it relies on a strong assumption: the attacker has full access to the target model, including its parameters and the ability to perform gradient computation. In many real-world scenarios, this assumption is unrealistic. Models deployed in production are typically exposed only through an interface (API) returning predictions, without revealing the internal architecture or the learned weights.

A *black-box* attack refers to any method by which an attacker attempts to mislead a model without knowledge of its internal structure, having access only to the inputs and outputs of the system:

$$x \longrightarrow f(x),$$

with no information about how f is computed, nor access to gradients. In this scenario, the attacker can no longer build a perturbation based on $\nabla_x J$ and must design alternative strategies to exploit vulnerabilities indirectly through observation.

Objective The goal of this part is to investigate to what extent it is possible to obtain effective adversarial examples in a realistic setting where gradients are not available. In particular, we aim to answer several questions: is it possible to reproduce the decision behavior of a target model by querying it? can this approximation be used to generate transferable adversarial perturbations? and under which conditions do attacks crafted on an external model successfully transfer to the original one?

Adopted strategy Several families of black-box attacks have been proposed in the literature, including query-based methods, gradient estimation techniques, and transfer-based approaches. In this project, we adopt an approach based on the construction of a *surrogate model*. The idea is to query the target model on a large number of inputs, collect the returned predictions, and then train a secondary model to imitate this behavior. Once this surrogate model is trained, we can again apply white-box methods such as FGSM or PGD on the surrogate and evaluate whether the resulting adversarial examples transfer to the target model.

This strategy allows us to combine the understanding gained in the previous section (local access to gradients) with a constrained access setting closer to real deployment conditions. The following sections describe each step in detail: extraction of data from the target model, surrogate training, fidelity evaluation, and finally adversarial generation and transfer.

7.2 Threat model and access assumptions

In a black-box setting, the interaction between the attacker and the target model is limited to queries submitted to the system, typically through an API. The attacker provides an input x and receives from the model either a predicted class $\hat{y}$ or a probability vector over the possible classes. No information regarding the internal architecture, parameters, or training data of the model is disclosed. This section formalizes the threat model considered in our experiments and specifies the access assumptions under which the attacks are conducted.

Target model We consider a target model f trained on the SST-2 sentiment classification task. The attacker has no knowledge of the internal structure of f , including:

- the architecture of the underlying neural network,
- the learned weights,
- the training procedure or hyperparameters,
- the format and content of the training dataset.

From the attacker’s perspective, f is a closed system representing a deployed service.

Available information The attacker is allowed to submit queries to f for arbitrary text inputs. Depending on the API configuration, two access levels are typically considered:

1. **Label-only access:** the system returns only the predicted class $\hat{y} = \arg \max f(x)$.
2. **Score-based access:** the system returns the probability vector $f(x)$ over the two sentiment classes.

In this work, we assume score-based access, as it allows the surrogate model to approximate not only the hard decisions of the classifier but also its confidence distribution.

No gradient access Crucially, the attacker does not have access to the gradients of the loss function with respect to the input:

$$\nabla_x J(\theta, x, y) \text{ is unavailable.}$$

As a consequence, classical white-box methods such as FGSM or PGD cannot be applied directly to the target model. Any gradient-based attack must therefore rely on an alternative estimation process or on a different model.

Query budget Black-box attacks often rely on sending a potentially large number of queries to the target model, which may be restricted in real-world settings due to rate limiting, cost, or detection. We denote by Q the maximum number of queries allowed during the attack. In this project, we consider a moderate query budget, sufficient to collect a representative dataset for surrogate training, without assuming unlimited access.

Attacker’s knowledge and capabilities The attacker is assumed to have the following capabilities:

- generate arbitrary input texts,
- observe the output returned by the API,
- collect a dataset of pairs $(x, f(x))$,
- train a surrogate model $\tilde{f}$ based on this dataset,
- apply white-box attacks to $\tilde{f}$.

However, the attacker *cannot*:

- compute gradients on the target model f ,
- inspect or modify the internal parameters,
- access internal representations such as embeddings or attention weights,
- influence the training data or fine-tune the target model.

Attack objective The goal of the attacker is to construct adversarial examples x_{adv} such that:

$$f(x_{\text{adv}}) \neq f(x),$$

while keeping x_{adv} as close as possible to the original input x according to a perturbation budget ϵ . The challenge lies in identifying perturbations that lead to misclassification *without* information about the internal decision boundary of the target model.

This threat model corresponds to a realistic deployment scenario where machine learning services are accessed remotely through an API, and where attackers attempt to manipulate outcomes without privileged access.

7.3 Attack I: Word substitution attack

7.3.1 Motivation

Before attempting more advanced transfer-based attacks, we first explored a simple black-box strategy operating directly in the input text space. The goal was to evaluate whether small, human-readable modifications of the sentence could induce a change in the prediction of the target model without any knowledge of its internal parameters. This class of attacks is common in natural language processing and aims to create adversarial examples by replacing a small number of words with semantically related alternatives.

7.3.2 Principle of the attack

In a label-only black-box setting, the attacker has access only to the predicted class $\hat{y}$ returned by the model:

$$x \rightarrow \hat{y} = \arg \max f(x),$$

with no probability scores and no gradient information. As a result, the attack cannot rely on a continuous optimization process. Instead, the adversary must perturb the original text in a discrete way, typically by substituting a small number of influential words and checking whether the output class changes.

Given an input sentence $x = (w_1, \dots, w_n)$, the attacker selects up to k words to replace with alternatives drawn from a predefined synonym dictionary:

$$x_{\text{adv}} = (w_1, \dots, w'_i, \dots, w_n),$$

with the objective that $f(x_{\text{adv}}) \neq f(x)$ while keeping x_{adv} linguistically plausible. The attacker evaluates each candidate by querying the target model and stops as soon as a successful adversarial example is found.

7.3.3 Implementation

We implemented a simple substitution-based attack in Python. A small dictionary of lexical replacements was manually defined, mapping sentiment-bearing words to neutral or opposite expressions (e.g., “amazing” $\rightarrow$ “okay”, “awesome”, “quite normal”; “terrible” $\rightarrow$ “not great”, “pretty bad”). The attack proceeds iteratively through the sentence and attempts to replace up to `max_modifications` words. For each modified text, the model is queried again.

A simplified version of the attack algorithm is shown below:

```
def black_box_word_substitution_attack(text, max_modifications=3):
    words = text.split()
    current_text = text
    original_label = oracle_predict(current_text)

    modifications = 0
    for i, w in enumerate(words):
        if w in synonym_dict and modifications < max_modifications:
            for candidate in synonym_dict[w]:
                new_words = words.copy()
                new_words[i] = candidate
                new_text = " ".join(new_words)

                new_label = oracle_predict(new_text)
                if new_label != original_label:
                    return new_text, new_label

            modifications += 1

    return current_text, original_label
```

Listing 6: Simplified word substitution attack

7.3.4 Results

We tested the attack on several sentences from the SST-2 dataset. In most cases, naive substitution did not cause the model to change its prediction, even when replacing multiple sentiment-heavy words. For instance, the sentence:

"This movie was amazing and awesome, I really loved it, such a great experience."

remained classified as “positive” after replacing words such as “amazing” and “awesome” with weaker expressions (e.g., “okay”, “quite normal”). This suggests that the classifier relies on a broader contextual signal, and not only on isolated keywords. It also highlights the limitations of a static synonym dictionary, which does not control for semantic coherence or syntactic fluidity.

The results of this first experiment were therefore limited. The attack succeeded rarely and required multiple artificial substitutions, sometimes degrading the grammatical and semantic quality of the sentence. As noted in our observations, this approach does not preserve the original meaning and does not constitute a realistic adversarial example.

7.3.5 Discussion

This experiment demonstrates that naive word substitution is not sufficient to reliably attack a modern transformer-based sentiment classifier. Several reasons explain this outcome:

- the contextual embeddings used by BERT capture sentiment at the phrase level, reducing dependence on a single token,
- simplistic synonym lists fail to preserve meaning and may introduce unnatural phrasing,
- the attack explores only a very small part of the discrete input space,
- no importance ranking or scoring was used to prioritize influential tokens.

Although this strategy is fully black-box and requires no training or gradient information, its effectiveness is limited. This motivates the second stage of our evaluation, in which we adopt a transf

7.4 Attack II: Surrogate model, dataset extraction and distillation

7.4.1 Rationale

After observing the limitations of naive word substitution, we moved to a more structured black-box strategy based on *model stealing*. The core idea is to treat the target model as an oracle, query it on a large number of inputs, and use the collected input–output pairs to train a new model that mimics its behaviour. This new model, called a *surrogate*, is under full white-box control: we can compute gradients, access internal representations, and therefore apply FGSM/PGD on it. If adversarial examples crafted on the surrogate transfer to the original model, the attacker effectively obtains a powerful black-box attack.

7.4.2 Dataset extraction from the target model

We first build a *stolen dataset* by querying the target model on a subset of the SST-2 training sentences. Concretely, we load $N = 2000$ sentences from the GLUE/SST-2 training split and send them to the oracle:

```
from datasets import load_dataset

N_SAMPLES = 2000
raw_ds = load_dataset("glue", "sst2")
texts = raw_ds["train"]["sentence"][:N_SAMPLES]

print("Example:", texts[0])
print("Nb texts used:", len(texts))
```

Listing 7: Building a stolen dataset from oracle queries

The oracle is the previously fine-tuned DISTILBERT model, now treated as a black-box API. We define helper functions to obtain softmax probabilities and hard labels:

```
import torch.nn.functional as F

def oracle_predict_proba(texts):
    if isinstance(texts, str):
        texts = [texts]
    enc = tokenizer(
        texts,
        padding=True,
        truncation=True,
        max_length=128,
        return_tensors="pt"
    ).to(DEVICE)

    with torch.no_grad():
        outputs = model(**enc)
        probs = F.softmax(outputs.logits, dim=-1)

    return probs.cpu()

def oracle_predict_label(texts):
    probs = oracle_predict_proba(texts)
    return probs.argmax(dim=-1)
```

Listing 8: Oracle prediction API

We then query the oracle in batches and store the *full probability vectors* rather than only the hard labels:

```
from tqdm.auto import tqdm

all_labels = []
BATCH = 32

for i in tqdm(range(0, len(texts), BATCH)):
    batch = texts[i:i+BATCH]
    preds = oracle_predict_proba(batch)      # shape: (batch_size, 2)
    all_labels.extend(preds.tolist())

len(all_labels)
```

Listing 9: Collecting soft labels from the oracle

The stolen dataset is then built as a Hugging Face **Dataset** where each example contains the original sentence and the corresponding oracle probabilities:

```
from datasets import Dataset

stolen_ds = Dataset.from_dict({
    "sentence": texts,
    "label": all_labels      # soft labels: [p_neg, p_pos]
})

stolen_ds
```

Listing 10: Stolen dataset construction

At this stage, we have a synthetic training set $\{(x_i, p_i)\}_{i=1}^N$ where $p_i = f(x_i)$ are the soft targets provided by the black-box model.

7.4.3 Tokenization and surrogate training

The stolen dataset is tokenized in the same way as the original SST-2 examples, and split into training and evaluation subsets:

```
def tokenize_batch(batch):
    return tokenizer(
        batch["sentence"],
        truncation=True,
        padding="max_length",
        max_length=128
    )

tokenized_stolen_ds = stolen_ds.map(tokenize_batch, batched=True)
tokenized_stolen_ds = tokenized_stolen_ds.rename_column("label", "labels")
tokenized_stolen_ds.set_format(
    type="torch",
    columns=["input_ids", "attention_mask", "labels"]
)

split = tokenized_stolen_ds.train_test_split(test_size=0.1, seed=42)
train_ds = split["train"]
eval_ds = split["test"]
```

Listing 11: Tokenization and split

We use the same architecture as the target, namely `distilbert-base-uncased` with a binary classification head, but this time trained to match the oracle's probabilities rather than the ground-truth SST-2 labels. This corresponds to a form of *knowledge distillation*:

```
import torch
from transformers import AutoModelForSequenceClassification, Trainer,
    TrainingArguments
import evaluate

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
SURROGATE_NAME = "distilbert-base-uncased"

surrogate_model = AutoModelForSequenceClassification.from_pretrained(
    SURROGATE_NAME,
    num_labels=2
).to(DEVICE)

accuracy_metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred

    # labels are soft (probabilities), we recover hard labels via argmax
    if labels.ndim > 1:
        labels = labels.argmax(axis=-1)

    preds = logits.argmax(axis=-1)
    return accuracy_metric.compute(predictions=preds, references=labels)

training_args = TrainingArguments(
    output_dir="./surrogate_results",
    num_train_epochs=3,
    per_device_train_batch_size=16,
```

```

        evaluation_strategy="epoch",
        report_to="none",
    )

    trainer = Trainer(
        model=surrogate_model,
        args=training_args,
        train_dataset=train_ds,
        eval_dataset=eval_ds,
        compute_metrics=compute_metrics,
    )

    trainer.train()
    trainer.evaluate()

```

Listing 12: Surrogate model and training loop

Soft labels vs. accuracy metric As highlighted in the notebook, the stolen dataset stores *full probability vectors* (`[p_neg, p_pos]`) as labels instead of hard class indices 0/1. During training, the surrogate sees these vectors as soft targets, which encourages it to reproduce not only the final decision but also the confidence profile of the oracle. However, the **accuracy** metric expects integer labels, so in `compute_metrics` we apply an **argmax** both to the surrogate logits and to the oracle probabilities before computing the agreement.

On the size of the stolen dataset. The surrogate was trained on $N = 2000$ samples extracted from the SST-2 training set through oracle queries. Although this number may appear small compared to the full corpus, it is a realistic constraint under a black-box threat model. In practice, a deployed model is accessed through an API, where each query has a monetary cost and a rate limit. The attacker must therefore balance fidelity against the total number of requests.

The choice $N = 2000$ corresponds to a *low-budget attack*: a limited number of queries, no parallel execution, and no prior access to internal representations. Despite this restricted setting, the surrogate reaches a decision agreement of 93.46% on unseen validation sentences, showing that even a relatively small number of samples is sufficient to recover most of the original decision boundary. This observation is consistent with prior work on model stealing, which shows that neural networks are highly query-efficient when the attacker observes full probability vectors instead of hard labels.

7.4.4 Measuring fidelity: agreement and KL divergence

To evaluate how well the surrogate model mimics the target model, we measure its fidelity on a held-out validation set. We use the SST-2 validation sentences as unseen inputs and compare the outputs of the oracle and the surrogate.

We first define a prediction function for the surrogate:

```

import torch.nn.functional as F

def surrogate_predict_proba(texts):
    if isinstance(texts, str):
        texts = [texts]
    enc = tokenizer(
        texts,
        padding=True,
        truncation=True,
        max_length=128,
        return_tensors="pt"
    )

```

```

).to(DEVICE)

with torch.no_grad():
    outputs = surrogate_model(**enc)
    probs = F.softmax(outputs.logits, dim=-1)

return probs.cpu()

def surrogate_predict_label(texts):
    probs = surrogate_predict_proba(texts)
    return probs.argmax(dim=-1)

```

Listing 13: Surrogate prediction API

We then compute, on all validation sentences, both the *agreement* (percentage of identical labels) and the average Kullback–Leibler (KL) divergence between the probability distributions:

```

from tqdm.auto import tqdm
import torch
import torch.nn.functional as F

val_texts = raw_ds["validation"]["sentence"]
print("Nb validation sentences:", len(val_texts))

BATCH = 32
oracle_probs_list = []
surrogate_probs_list = []

for i in tqdm(range(0, len(val_texts), BATCH)):
    batch = val_texts[i:i+BATCH]
    oracle_probs_list.append(oracle_predict_proba(batch))
    surrogate_probs_list.append(surrogate_predict_proba(batch))

oracle_probs = torch.cat(oracle_probs_list, dim=0)
surrogate_probs = torch.cat(surrogate_probs_list, dim=0)

oracle_labels = oracle_probs.argmax(dim=-1)
surrogate_labels = surrogate_probs.argmax(dim=-1)

agreement = (oracle_labels == surrogate_labels).float().mean().item()
print(f"Agreement oracle vs surrogate : {agreement:.4f}")

kl = F.kl_div(
    surrogate_probs.log(), # log Q (surrogate)
    oracle_probs,          # P (oracle)
    reduction="batchmean"
).item()
print(f"KL(P_oracle || P_surrogate) : {kl:.6f}")

```

Listing 14: Agreement and KL between oracle and surrogate

The obtained agreement is:

$$\text{agreement} \approx 0.9346 \quad (93.46\%),$$

meaning that the surrogate reproduces more than 93% of the oracle’s decisions on sentences it has never seen during training. The average KL divergence between the two distributions is approximately:

$$\text{KL}(P_{\text{oracle}} \parallel P_{\text{surrogate}}) \approx 0.17,$$

which indicates that the probability distributions are very close. In other words, the surrogate does not only copy the hard labels, but also closely matches the confidence profile of the target model.

Agreement oracle vs surrogate : 0.9346
KL(P_oracle || P_surrogate) : 0.175711

7.4.5 Discussion

How to interpret these scores, the surrogate reaches an agreement of 93.46 % with the oracle on unseen sentences and a KL divergence of approximately 0.17 between their output distributions. The first number indicates how often both models predict the same label, while the second measures how close their full probability distributions are. Tables 3 and 4 summarise typical ranges of values.

Agreement	Interpretation
< 70 %	failed theft
70–85 %	medium-quality theft
85–95 %	successful theft
> 95 %	near-perfect clone

Table 3: Qualitative interpretation of the agreement between the oracle and the surrogate. Our score of 93.46 % falls in the “successful theft” range.

KL divergence	Typical interpretation
0.0	identical distributions (perfect clone)
0.1–0.3	very close distributions
1.0–5.0	noticeably different
≈ 10	almost completely different

Table 4: Qualitative interpretation of KL divergence between the oracle and the surrogate. Our value ($KL \approx 0.17$) lies in the “very close” regime, meaning the surrogate copies not only the labels but also the confidence of the target model.

These results show that the model stealing attack is successful: using only 2000 queries to the black-box model, the attacker is able to train a surrogate that reproduces more than 9 out of 10 decisions of the target and closely approximates its probability outputs. From the attacker’s perspective, this surrogate can now be treated as a white-box model: gradients are available, internal representations can be accessed, and classical attacks such as FGSM and PGD can be applied.

In the remainder of the black-box study, this surrogate model serves as a proxy to craft adversarial examples in a white-box fashion and to study how well these examples transfer back to the original target model.

7.5 Attack III : Adversarial attacks through the stolen model

The previous section established that the surrogate model $\tilde{f}$ closely matches the oracle f through agreement and KL divergence metrics. We now exploit this white-box access to $\tilde{f}$ to craft adversarial examples, and evaluate whether these perturbations *transfer* to the original API, which remains a black-box model. This corresponds to a realistic threat model: the attacker

never accesses gradients of f , but can still perform exhaustive search or gradient-based attacks on the surrogate $\tilde{f}$.

7.5.1 Experimental protocol: selecting aligned samples

Before launching any attack, we restrict ourselves to sentences on which the oracle and the surrogate initially *agree*. This avoids artefacts where an “attack” would only exploit an existing disagreement between the two models.

We first reuse the prediction helpers defined for the surrogate:

```
import torch
import torch.nn.functional as F

def surrogate_predict_proba(texts):
    if isinstance(texts, str):
        texts = [texts]
    enc = tokenizer(
        texts,
        padding=True,
        truncation=True,
        max_length=128,
        return_tensors="pt"
    ).to(DEVICE)

    with torch.no_grad():
        outputs = surrogate_model(**enc)
        probs = F.softmax(outputs.logits, dim=-1)
    return probs.cpu()

def surrogate_predict_label(texts):
    probs = surrogate_predict_proba(texts)
    return probs.argmax(dim=-1)
```

Listing 15: Surrogate prediction API used in the attack phase

We then pick the first 200 validation sentences from SST-2 and keep only those for which both models predict the same label:

```
from tqdm.auto import tqdm
import numpy as np

# Take for instance 200 validation sentences
all_val_texts = raw_ds["validation"]["sentence"][:200]

oracle_labels_all = oracle_predict_label(all_val_texts).numpy()
surrogate_labels_all = surrogate_predict_label(all_val_texts).numpy()

agree_mask = oracle_labels_all == surrogate_labels_all
agree_texts = [t for t, m in zip(all_val_texts, agree_mask) if m]
agree_oracle_labels = oracle_labels_all[agree_mask]
agree_surrogate_labels = surrogate_labels_all[agree_mask]

print("Nb phrases validation totales :", len(all_val_texts))
print("Nb phrases o  oracle et surrogate sont d'accord :", len(
    agree_texts))

# We attack a small subset for speed
N_ATTACK_SAMPLES = 30
attack_texts = agree_texts[:N_ATTACK_SAMPLES]
```

```

attack_oracle_labels = agree_oracle_labels[:N_ATTACK_SAMPLES]
attack_surrogate_labels = agree_surrogate_labels[:N_ATTACK_SAMPLES]

```

Listing 16: Selecting validation sentences where oracle and surrogate agree

This protocol ensures that any label flip observed after perturbation is not due to an initial mismatch between f and $\tilde{f}$, but to the attack itself. The subset of 30 sentences is a compromise between computational cost and diversity of cases for qualitative analysis.

7.6 Attack III Phase I: adversarial examples by word deletion

7.6.1 Method

The first attack is deliberately simple: we try to make the surrogate change its prediction by deleting *exactly one* word from the input. Given a sentence $x = (w_1, \dots, w_n)$ and its surrogate label $\tilde{y} = \arg \max \tilde{f}(x)$, we generate candidates:

$$x_{-i} = (w_1, \dots, w_{i-1}, w_{i+1}, \dots, w_n), \quad i = 1, \dots, n,$$

and search for a position i such that:

$$\arg \max \tilde{f}(x_{-i}) \neq \tilde{y}.$$

The corresponding Python function is:

```

def generate_adversarial_by_deletion(text, orig_label_surrogate):
    """
    Try to delete a single word from 'text' to change the surrogate
    prediction.
    Returns (adv_text, removed_index) if found, otherwise (None, None).
    """
    words = text.split()
    if len(words) <= 1:
        return None, None # not enough words to delete

    for i in range(len(words)):
        # Construct a sentence where word i is deleted
        new_words = words[:i] + words[i+1:]
        new_text = " ".join(new_words)
        new_label = surrogate_predict_label(new_text).item()

        if new_label != orig_label_surrogate:
            return new_text, i # adversarial found

    return None, None

```

Listing 17: Generating adversarial examples by single-word deletion

We then apply this search to each of the 30 selected sentences, and record not only the surrogate behaviour but also the oracle behaviour on the same perturbed text:

```

adversarial_examples = []

for text, lab_o, lab_s in tqdm(
    zip(attack_texts, attack_oracle_labels, attack_surrogate_labels)
    ,
    total=len(attack_texts)):
    adv_text, removed_index = generate_adversarial_by_deletion(text,
        lab_s)

```

```

if adv_text is None:
    # No adversarial example found for the surrogate
    adversarial_examples.append({
        "original_text":      text,
        "adv_text":           None,
        "orig_label_oracle":  int(lab_o),
        "orig_label_surrogate": int(lab_s),
        "adv_label_surrogate": None,
        "adv_label_oracle":   None,
        "success_surrogate":  False,
        "success_oracle":     False
    })
else:
    # Evaluate the oracle as well on the adversarial text
    adv_lab_s = surrogate_predict_label(adv_text).item()
    adv_lab_o = oracle_predict_label(adv_text).item()

    adversarial_examples.append({
        "original_text":      text,
        "adv_text":           adv_text,
        "orig_label_oracle":  int(lab_o),
        "orig_label_surrogate": int(lab_s),
        "adv_label_surrogate": int(adv_lab_s),
        "adv_label_oracle":   int(adv_lab_o),
        "success_surrogate":  (adv_lab_s != lab_s),
        "success_oracle":     (adv_lab_o != lab_o)
    })

```

Listing 18: Applying word deletion attack to both surrogate and oracle

7.6.2 Metrics and quantitative results

We store the results in a `pandas` dataframe and compute: (i) the attack success rate on the surrogate, (ii) the attack success rate on the oracle, and (iii) the transfer rate (success on both given success on the surrogate):

```

import pandas as pd

df_adv = pd.DataFrame(adversarial_examples)

# Attack success on surrogate = label changed
success_surrogate = df_adv["success_surrogate"].sum()
success_oracle    = df_adv["success_oracle"].sum()
n = len(df_adv)

attack_success_rate_surrogate = success_surrogate / n
attack_success_rate_oracle    = success_oracle / n

# Transfer rate: among attacks that succeed on the surrogate,
# how many also succeed on the oracle?
if success_surrogate > 0:
    transfer_rate = df_adv[
        df_adv["success_surrogate"] & df_adv["success_oracle"]
    ].shape[0] / success_surrogate
else:
    transfer_rate = 0.0

```

```

print(f"Nb phrases attaquées : {n}")
print(f"Success attaque sur surrogate : {success_surrogate} ({
    attack_success_rate_surrogate:.2%})")
print(f"Success attaque sur oracle : {success_oracle} ({
    attack_success_rate_oracle:.2%})")
print(f"Taux de transfert : {transfer_rate:.2%}")

```

Listing 19: Computing success and transfer rates for the deletion attack

On our run, we obtain:

Nb phrases attaquées = 30, Succès_f = 8 (26.67%), Succès_f = 7 (23.33%), Transfer = 87.50%.

In other words, roughly a quarter of the sentences can be flipped on the surrogate by deleting a single word, and most of these successful attacks also transfer to the oracle.

7.6.3 Qualitative examples

The following examples are particularly illustrative: deleting a single word is enough to reverse the sentiment prediction for both models:

```

print("\nExemples o l'attaque réussit sur surrogate ET oracle :\n")
both_success = df_adv[df_adv["success_surrogate"] & df_adv["
    success_oracle"]].head(5)

for idx, row in both_success.iterrows():
    print("-" * 80)
    print("Texte original :")
    print(row["original_text"])
    print(f"Labels init (oracle, surrogate) : ({row['orig_label_oracle
        ']}, {row['orig_label_surrogate']})")
    print("\nTexte adversarial :")
    print(row["adv_text"])
    print(f"Labels adv (oracle, surrogate) : ({row['adv_label_oracle
        ']}, {row['adv_label_surrogate']})")
    print()

```

Listing 20: Examples of successful deletion attacks on both surrogate and oracle

A few representative cases are:

- **Original:** *“a sometimes tedious film .”* → negative (0) for both models.
Adversarial (delete “tedious”): *“a sometimes film .”* → positive (1) for both models.
- **Original:** *“in its best moments , resembles a bad high school production of grease , without benefit of song .”*
Adversarial (delete “bad”): *“in its best moments , resembles a high school production of grease , without benefit of song .”*
 Here the explicit negative cue “bad” is removed, turning the sentence into a mildly positive or neutral description.
- **Original:** *“even horror fans will most likely not find what they ’re seeking with trouble every day ; the movie lacks both thrills and humor .”*
Adversarial: *“even horror fans will most likely not find what they ’re seeking with trouble every day ; the movie both thrills and humor .”*
 Deleting “lacks” completely reverses the meaning of the final clause.

7.6.4 Critical discussion

From a purely numerical point of view, the attack seems effective: we obtain a non-negligible success rate on the surrogate and a high transfer rate to the oracle. However, a closer qualitative inspection reveals several important limitations:

- In almost all successful cases, the deleted token is a *sentiment pivot* such as “*bad*”, “*lacks*”, “*tedious*” or expressions associated with pity and disappointment. These words carry a strong semantic weight in a sentiment classification task. It is therefore expected and *not surprising* that removing them flips the label.
- In some examples, the deletion not only removes a negative cue but also produces a sentence whose meaning is *genuinely* different for a human. For instance, removing “*lacks*” from “*lacks both thrills and humor*” yields the opposite assertion. This is closer to an *editing* of the opinion than to a minimal perturbation.
- The attack does not enforce any constraint on grammaticality or semantic preservation. Some outputs become slightly awkward or incomplete, yet are still interpretable by the models.

As a consequence, these deletion-based examples do *not* reveal a subtle vulnerability of BERT to imperceptible perturbations. Instead, they highlight a more trivial property: both models are strongly dependent on a small set of highly informative tokens, and drastically changing these tokens naturally changes the predicted sentiment. From an auditing perspective, this is still informative (it confirms the reliance on sentiment pivots), but it is closer to a sanity check than to a sophisticated adversarial attack.

7.7 Attack III phase II: minimal perturbation by symbol insertion

7.7.1 Motivation

To move towards more “minimal” perturbations, we then tested an attack that leaves the textual content unchanged and only appends a neutral symbol at the end of the sentence. For example:

“This movie is great.” → “This movie is great. ”.

For a human reader, the sentiment is identical; the symbol is purely decorative. If such a modification were to flip the classifier’s output, it would indicate a lack of robustness to superficial formatting changes or spurious correlations learned during training.

7.7.2 Method and implementation

We define a small set of symbols:

```
symbols = [" ", "!", "~", " ", " ", " ", " ", " ", " ", " ", " "]
results_symbols = []

for text, lab_o, lab_s in zip(attack_texts, attack_oracle_labels,
                             attack_surrogate_labels):
    for sym in symbols:
        adv_text = text + " " + sym
        adv_lab_s = surrogate_predict_label(adv_text).item()
        adv_lab_o = oracle_predict_label(adv_text).item()

        results_symbols.append({
            "symbol": sym,
```

```

        "original":          text,
        "adv_text":         adv_text,
        "orig_label":       int(lab_s),
        "adv_label_surrogate": int(adv_lab_s),
        "adv_label_oracle": int(adv_lab_o),
        "success_surrogate": adv_lab_s != lab_s,
        "success_oracle":   adv_lab_o != lab_o
    })

df_symbols = pd.DataFrame(results_symbols)

# Global statistics
succ_s = df_symbols["success_surrogate"].sum()
succ_o = df_symbols["success_oracle"].sum()
total  = len(df_symbols)

print(f"Succ s surrogate (symbol attack) : {succ_s}/{total} = {succ_s/total:.2%}")
print(f"Succ s oracle      (symbol attack) : {succ_o}/{total} = {succ_o/total:.2%}")

```

Listing 21: Testing neutral symbol insertion

We obtain $N = 30 \times 9 = 270$ trials in total (30 sentences, 9 symbols). In our experiments, the success rates were extremely low for both models (typically close to 0%), and no meaningful adversarial pattern could be identified.

7.7.3 Critical discussion

The failure of the symbol insertion attack is, in this case, a *reassuring* result:

- The model does not overreact to purely decorative tokens and seems robust to this type of superficial noise.
- The lack of success indicates that the embeddings and attention mechanisms of DISTIL-BERT have not memorised spurious correlations between such Unicode symbols and sentiment labels on SST-2.
- From a methodological point of view, this negative result is important to report: it shows that not every naive perturbation qualifies as an adversarial vulnerability, and that robust models may simply ignore irrelevant tokens.

In combination with the deletion-based attack, this experiment suggests the following picture: the model is stable under very small, semantically neutral perturbations (symbols), but highly sensitive to the presence or absence of a few key sentiment words.

7.8 Attack IV: Contextual masked-LM substitution attack

The previous two experiments covered two extremes of textual perturbation:

(1) removing a full word, which often results in substantial semantic loss, and (2) adding decorative Unicode symbols, which preserves meaning but fails to influence the model. As a more realistic adversarial scenario, we now consider *contextual synonym substitution* guided by a masked language model (MLM). The goal is to minimally alter the lexical surface of a sentence while preserving its semantic content, hoping to induce a different prediction from the classifier.

Principle. Given an input sentence x and the surrogate model $\tilde{f}$, we seek a perturbation x' such that:

$$\tilde{f}(x) = y, \quad \tilde{f}(x') \neq y,$$

while enforcing:

$$\text{Semantics}(x') \approx \text{Semantics}(x),$$

i.e. x' should be close to x in meaning. To achieve this, we rank words in x based on their *importance* to the prediction: removing a highly influential word should reduce the confidence of the surrogate model. We then replace the top- k important words using an MLM (DistilBERT in masked-LM mode), which proposes contextually appropriate substitutes for the masked token.

Word importance. For each token position i , we measure its contribution as the drop in the surrogate’s confidence when removing the word:

$$s_i = p(y \mid x) - p(y \mid x_{\setminus i}),$$

where $x_{\setminus i}$ is the sentence without word i . This results in a simple and computationally tractable importance score.

```
def word_importance_scores(text, orig_label_surrogate):
    words = text.split()
    probs_orig = surrogate_predict_proba(text)[0].numpy()
    p_orig = probs_orig[orig_label_surrogate]

    scores = []
    for i in range(len(words)):
        new_text = " ".join(words[:i] + words[i+1:])
        p_new = surrogate_predict_proba(new_text)[0].numpy()[
            orig_label_surrogate]
        scores.append(p_orig - p_new)
    return scores
```

Listing 22: Word importance via probability drop

Contextual substitution. We mask the selected word and use the MLM to sample the k most probable contextual candidates, discarding substitutions that either (i) preserve the surrogate’s original label, or (ii) significantly alter the sentence meaning. The latter constraint is enforced by measuring the cosine similarity between the [CLS] embeddings of the original and modified sentences produced by the surrogate model. Only candidates with similarity above a threshold (e.g. ≥ 0.85) are retained. We chose this threshold after a few experiments with higher values that produced no results.

```
masked = text.replace(words[idx], MASK_TOKEN)
candidates = fill_mask(masked, top_k=10)

for cand in candidates:
    cand_text = cand["sequence"].strip()
    if surrogate_predict_label(cand_text).item() != orig_label:
        sim = cosine_similarity_sentences(text, cand_text)
        if sim >= 0.85:
            return cand_text      # adversarial candidate
```

Listing 23: Contextual replacement using a masked LM

Results. We apply this attack to 60 validation sentences on which the surrogate and oracle agree. This value makes the attack time grow to around 20 minutes. The outcome is:

$$\text{Success (surrogate)} = 8/60 = 13.33\%, \quad \text{Success (oracle)} = 2/60 = 3.33\%.$$

Thus, the contextual attack is able to generate adversarial examples that occasionally fool both the surrogate and the oracle, although flip rates remain moderate.

Example of an attacked sentence:

"a **synthesis/series** of cliches and absurdities that seems positively decadent in its cinematic flash and emptiness ."

"Synthesis" -> 1.0 (positive) "Series" -> 0.0 (negative) Similarity = 0.877

Critical analysis. At first sight, the attack “fails”. In practice, this negative result is *informative* for several reasons:

- The surrogate appears *robust* to minimal lexical changes: replacing a single word with a contextually plausible synonym does not change its decision on SST-2 sentiment classification. This is consistent with the nature of the task, where sentiment is often expressed redundantly through multiple cues.
- The semantic constraint (high cosine similarity between embeddings) enforces a strong notion of content preservation. Many potentially adversarial substitutions are rejected because they alter the meaning too much. Relaxing this threshold would increase the success rate, at the cost of producing less faithful adversarial examples.
- Compared to word deletion, masked substitution is a *harder* attack: deletion removes most of the sentiment signal when targeting words such as “boring”, “tedious”, or “bad”, whereas substitution often preserves the overall polarity of the sentence. The model’s sensitivity observed in the deletion experiment does not trivially translate to sensitivity under subtle paraphrasing.
- Methodologically, this highlights the gap between *semantic-preserving* adversarial attacks and simple perturbation heuristics. Naively removing or altering a single word is often insufficient to cross a decision boundary in high-dimensional embedding space when attention models integrate multiple clues.

In short, the lack of success does not indicate an implementation error, but rather illustrates a desirable property of the model at this level of perturbation: sentiment decisions are not dominated by one token alone when plausible synonyms are used. More sophisticated attacks (e.g. multi-step contextual substitution, gradient-guided synonym search, or paraphrasing with semantic constraints) would likely increase the success rate, and constitute a natural extension of this experiment.

7.9 Attack V: Membership Inference Attack (Black-Box)

7.9.1 Motivation and Threat Model

So far, our black-box study focused on the functional vulnerabilities of the target model (adversarial instability, backdoors, data extraction). A complete audit should also investigate potential privacy leakage. A Membership Inference Attack (MIA) seeks to determine whether a specific input x was part of the model’s fine-tuning dataset.

We assume a *pure black-box threat model*: the adversary can send any text input to the target API and retrieve its softmax probability vector, but has no access to weights, gradients, losses, or internal representations.

The question is therefore simple:

Do confidence scores contain enough signal to guess if the text was seen during training?

7.9.2 Experimental Setup

To simulate a real attack, we build two datasets:

- **Members (train):** $N = 1000$ sentences from the SST-2 training split,
- **Non-members (val):** $N' = 872$ sentences from the SST-2 validation split.

Each text x_i is queried through the oracle API to collect

$$f(x_i) = \text{softmax}(\text{logits}(x_i)) \in \mathbb{R}^2.$$

The attack model receives only this probability vector and must classify samples as **member** or **non-member**.

We extract three classical features used in the literature:

- *Max probability:* $\max(f(x))$,
- *Margin:* $\max(f(x)) - \text{second-highest}(f(x))$,
- *Entropy:* $-\sum_j f_j(x) \log f_j(x)$.

These statistics capture overconfidence patterns associated with memorization: models tend to assign sharper probability distributions to samples they have been trained on.

7.9.3 Implementation

Querying the oracle. The following helper collects softmax scores in batches:

```
def get_probs_in_batches(texts, predict_proba_fn, batch_size=32):
    all_probs = []
    for i in range(0, len(texts), batch_size):
        batch = texts[i:i+batch_size]
        probs = predict_proba_fn(batch)          # Tensor (B, 2)
        all_probs.append(probs)
    return torch.cat(all_probs, dim=0).numpy()
```

Listing 24: Collecting probability vectors from the oracle

We obtain two sets of probability vectors:

```
probs_train = get_probs_in_batches(train_texts, oracle_predict_proba)
probs_val    = get_probs_in_batches(val_texts,    oracle_predict_proba)
```

Listing 25: Building the MIA dataset

Feature extraction.

```
def probs_to_features(probs):
    max_prob = probs.max(axis=1)
    sorted_p = np.sort(probs, axis=1)[:, ::-1]
    second   = sorted_p[:, 1]
    margin    = max_prob - second
    entropy   = -np.sum(probs * np.log(probs + 1e-12), axis=1)
    return np.stack([max_prob, margin, entropy], axis=1)
```

Listing 26: Feature extraction from softmax scores

We label train samples as 1 (members) and validation samples as 0 (non-members):

```
X_members = probs_to_features(probs_train)
X_nonmem  = probs_to_features(probs_val)

X = np.concatenate([X_members, X_nonmem], axis=0)
y = np.concatenate([
    np.ones(len(X_members), dtype=int),
    np.zeros(len(X_nonmem), dtype=int)
])
```

Listing 27: Final MIA dataset

Training the attack model. The attacker uses a simple baseline: standardisation + logistic regression.

```
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42
)

attack_model = make_pipeline(
    StandardScaler(),
    LogisticRegression(max_iter=1000)
)

attack_model.fit(X_tr, y_tr)

y_pred = attack_model.predict(X_te)
y_score = attack_model.predict_proba(X_te)[:, 1]

acc = accuracy_score(y_te, y_pred)
auc = roc_auc_score(y_te, y_score)
```

Listing 28: Training and evaluating the MIA attacker

7.9.4 Results

On the oracle API, the attacker obtains:

$$\text{Accuracy} = 0.6085, \quad \text{ROC-AUC} = 0.5709.$$

Metric	Value	Interpretation
Accuracy	0.61	+11 points above random guessing
ROC-AUC	0.57	weak discriminative ability

Table 5: Membership inference attack on the oracle API.

While slightly above 0.5, these values indicate that the attack model can only *weakly* separate members from non-members, suggesting that the oracle does not strongly memorize the SST-2 training set.

7.9.5 Critical Discussion

The membership inference signal is present, but weak. Several factors explain the limited success:

- **Pre-trained encoder.** DistilBERT already encodes sentiment patterns before fine-tuning, reducing the need to memorize individual examples.
- **Small downstream dataset.** SST-2 contains only a few thousand samples; most of the knowledge comes from pre-training, not from the fine-tuning stage.
- **Good generalisation.** The fine-tuning procedure does not overfit to specific training sentences, resulting in similar confidence patterns on unseen validation examples.

From a defensive perspective, this *negative result* is encouraging: the oracle does not trivially reveal training membership through confidence scores alone.

However, these experiments represent only a baseline attack: shadow model training, calibrated scores, or loss-based inference could yield stronger signals. Future work may explore these more sophisticated strategies.

7.10 Summary of the black-box attack phase

The most significant outcome of the black-box phase is the success of the *model stealing* attack. With only 2000 API queries, the surrogate $\hat{f}$ reproduces 93.46% of the oracle’s decisions on unseen validation data, and matches its confidence distribution with a Kullback–Leibler divergence of $\text{KL}(P_{\text{oracle}}\|P_{\text{surrogate}}) \approx 0.17$. In practical terms, the attacker is able to rebuild an almost identical classifier using nothing more than input–output observations, without ever accessing internal weights or gradients. This enables full white-box adversarial capabilities on a model that is otherwise deployed behind a closed API.

The experiments conducted on top of the surrogate lead to a nuanced view of adversarial vulnerability in the black-box setting:

- The *word deletion* attack transfers efficiently: whenever a deletion causes the surrogate to change its prediction, the oracle often follows. This indicates that the surrogate captures the decision boundary of the original model well enough to generate transferable adversarial inputs.
- However, the perturbation is not truly “minimal”: the removed tokens are semantically essential sentiment pivots (e.g., *bad*, *lacks*, *pitiful*), so the attack is less “adversarial” in the strict sense, and more a demonstration of sensitivity to key lexical cues.
- The *symbol insertion* attack, which corresponds to a genuinely minimal perturbation from a human perspective, does not succeed in this setup, suggesting that the model is robust to superficial noise and ignores purely decorative tokens.

Taken together, these results provide two important insights for model security. First, protecting internal parameters is not sufficient: a near-copy of a private model can be reconstructed from a limited number of API calls, enabling subsequent white-box attacks. Second, evaluating robustness requires semantically grounded perturbations rather than brute-force edits that drastically change the input. Future work should therefore combine our surrogate with state-of-the-art text attack methods (TextAttack, OpenAttack, etc.) to obtain a more comprehensive assessment of black-box robustness.

8 Conclusion

This project has allowed us to explore, in a concrete and experimental way, the intersection between modern NLP models and adversarial security. Our initial motivation was to understand how a widely deployed model such as DISTILBERT behaves not only under normal operating

conditions, but also when it is exposed to active attempts to reverse, steal or mislead its decision function.

From a technical perspective, the work progressed along two complementary axes. In the white-box setting, we gained full access to the model parameters and gradients, which enabled us to implement standard adversarial techniques such as FGSM and PGD directly in the embedding space. This phase taught us to reason about the geometry of the model’s loss landscape, and to observe experimentally how small perturbations in a continuous representation can induce large changes in the model output. We also discovered the importance of hyperparameters (step size, number of iterations, norm constraints), and how delicate it can be to balance attack strength and semantic preservation in text-based adversarial scenarios.

In the black-box phase, we explored the opposite threat model: a remote API with no access to internal weights. The key insight of this part is that “security through opacity” is not a sufficient defence: by treating the target model as an oracle, we successfully implemented a complete model stealing pipeline. With only 2 000 queries, we trained a surrogate model whose decisions match the oracle more than 93 % of the time on unseen texts, and whose probability distributions are close in the KL sense. This surrogate makes the “black box” effectively white-box for the attacker, since gradients are now available on a proxy model that behaves almost identically to the original system.

The final stage of the project was dedicated to adversarial attacks in the black-box context, using the surrogate as an offline experimentation tool. Although our attacks were intentionally simple (deletion of a single token, insertion of neutral symbols), we still observed a high transfer rate: most successful adversarial examples on the surrogate also misled the oracle. At the same time, our results highlighted the limits of naive perturbations: deleting sentiment-bearing tokens is not a “minimal” attack, and changing the meaning of the sentence is fundamentally different from subtle adversarial manipulation. These observations led us to reflect critically on what it means to evaluate robustness, and to distinguish between genuine adversarial vulnerabilities and simple semantic edits.

Lessons learned and perspectives

Beyond the direct results, this project was an opportunity to deepen our understanding of several key concepts in AI security:

- **Opacity is not security:** hiding the parameters of a model does not prevent an attacker from reproducing its behaviour.
- **Surrogate models are powerful tools:** once a surrogate exists, adversarial research becomes significantly easier and cheaper than live interaction with the target API.
- **Adversarial evaluation requires constraints:** attacks must be designed to preserve semantic content in order to reveal meaningful vulnerabilities.
- **Robustness is multi-dimensional:** a model may be robust to some perturbations (formatting, symbols) and sensitive to others (sentiment pivots), and only a systematic methodology can expose such nuances.
- **Research is iterative:** negative results (e.g., symbol insertion) are as valuable as positive ones, because they help refine intuition about model behaviour.

If we had more time, we would extend this work in several directions: leveraging dedicated adversarial NLP frameworks (TextAttack, OpenAttack), enforcing semantic constraints through similarity metrics, exploring paraphrasing-based attacks, investigating robustness beyond SST-2, and studying the impact of defensive techniques such as adversarial training or confidence calibration. In a broader context, our experiments raise interesting questions about the security

of NLP systems deployed in real-world applications, especially when access to a public API is sufficient to recover a high-fidelity clone.

Acknowledgements

We would like to thank you sir for the opportunity to work on this project and for the guidance provided throughout the course. The project required us to combine machine learning knowledge, experimental rigour, and practical software engineering skills, and we are grateful for the constructive feedback that helped shape our methodology.

The author:

Pierre-louis Tissot-Favre

9 References

Adversarial attacks in NLP

- Ebrahimi et al. (2018), *HotFlip: White-box adversarial examples for text classification*.
- Ren et al. (2019), *TextFooler: A text-based adversarial attack*.
- Garg & Ramakrishnan (2020), *BAE: BERT-based adversarial examples*.
- Li et al. (2019), *BERT-Attack: Adversarial attack against BERT using BERT*.
- Jia & Liang (2017), *Adversarial examples for evaluating reading comprehension systems*.
- Alzantot et al. (2018), *Generating natural language adversarial examples*.

Black-box model extraction

- Tramèr et al. (2016), *Stealing machine learning models via prediction APIs*.
- Orekondy et al. (2019), *Knockoff Nets: Stealing functionality of black-box models*.
- Sanyal et al. (2022), *Stealing Attention: Model extraction attacks against NLP models*.

Membership inference

- Shokri et al. (2017), *Membership inference attacks against machine learning models*.
- Salem et al. (2018), *ml-leaks: Model and data independent membership inference attacks*.
- Li et al. (2021), *Membership inference attacks against large language models*.

Foundational adversarial ML

- Goodfellow et al. (2015), *Explaining and harnessing adversarial examples*.
- Madry et al. (2018), *Towards deep learning models resistant to adversarial attacks*.

Online resources

- [YouTube video: Black-box / adversarial ML \(1\)](#)
- [YouTube video: Black-box / adversarial ML \(2\)](#)
- [YouTube video: Neural networks / deep learning intuition](#)
- [YouTube video: Machine learning security / attacks overview](#)
- [YouTube video: Black-box model attacks explained](#)
- [arXiv:2109.07403](#)
- [ETH Zurich student thesis on black-box ML security](#)
- [Medium article: Pen-testing black-box ML models \(visual guide\)](#)
- [SpecterOps blog: Attacking black-box models](#)
- [MDPI article \(Symmetry, Vol. 17, No. 7, 985\)](#)
- [PMC article on machine learning and security/privacy aspects](#)